LABORATORIUM 1

Metody reprezentacji wiedzy

Przedmiot	Sztuczna inteligencja
Forma	Laboratorium
Czas	4 h dydaktyczne (190 min), 20.10.2026
Grupy	INLS5_SI
Prowadzący	mgr inż. Szymon Guzik

Cele zajęć

- Wyjaśnić, czym jest reprezentacja wiedzy i po czym ocenia się jej jakość.
- Zbudować w Pythonie sieć semantyczną z dziedziczeniem i wyjątkami.
- Opisać obiekt ramką (frame) ze slotami, wartościami domyślnymi i demonem.
- Zapisać wiedzę jako fakty i reguły oraz wyprowadzić nowe fakty wnioskowaniem w przód.
- Wybrać temat projektu semestralnego i założyć jego repozytorium.



Reprezentacja wiedzy — ten sam świat opisany na kilka sposobów

1. Plan zajęć (190 minut)

Czas	Co robimy
0–20 min	Wprowadzenie: czym jest wiedza w systemie SI, kryteria dobrej reprezentacji

Czas	Co robimy
20–60 min	Sieci semantyczne — przykład, uruchomienie, zadania 1–2
60–70 min	Przerwa
70–110 min	Ramki (frames) — przykład, zadanie 3
110–150 min	Logika i reguły, wnioskowanie w przód — zadania 4–5
150–190 min	Projekt semestralny: wybór tematu, zespół, repozytorium

2. Teoria

Czym jest reprezentacja wiedzy

System sztucznej inteligencji nie „rozumie" świata — operuje na jego **modelu**. Reprezentacja wiedzy to sposób zapisania faktów, pojęć i zależności tak, żeby program mógł z nich **wnioskować**, czyli wyprowadzać informacje, których nikt mu wprost nie podał. Ten sam fragment rzeczywistości można opisać na wiele sposobów; wybór sposobu decyduje o tym, co system potrafi policzyć i jak szybko.

2.1. Kryteria dobrej reprezentacji

Kryterium	Pytanie kontrolne
Adekwatność reprezentacyjna	Czy da się zapisać wszystko, co jest potrzebne (także wyjątki i niepewność)?
Adekwatność wnioskowania	Czy z zapisu można wyprowadzić nową wiedzę?
Efektywność wnioskowania	Czy wnioskowanie kończy się w rozsądnym czasie?
Efektywność pozyskiwania	Czy ekspert lub program łatwo dopisze nową wiedzę?
Czytelność	Czy człowiek rozumie zapis i może go sprawdzić?

2.2. Przegląd metod

Metoda	Idea	Mocne strony	Słabe strony
Logika predykatów	Fakty i formuły, wnioskowanie formalne	Precyzja, dowodliwość	Trudna przy wyjątkach i niepewności

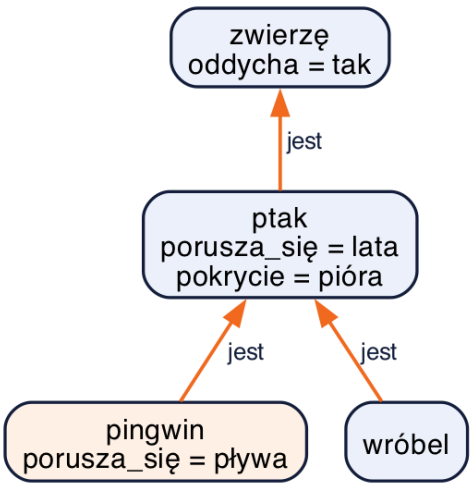
Metoda	Idea	Mocne strony	Słabe strony
Reguły produkcji (JEŻELI–TO)	Wiedza ekspercka jako reguły	Czytelne, łatwe do rozbudowy	Konflikty reguł, brak struktury
Sieci semantyczne	Graf pojęć i relacji	Naturalne dziedziczenie	Brak formalnej semantyki relacji
Ramki (frames)	Obiekt = zestaw slotów z wartościami	Wartości domyślne, procedury	Słabsze wnioskowanie ogólne
Ontologie (RDF/OWL)	Standaryzowane trójki i klasy	Wymiana wiedzy między systemami	Złożoność narzędzi
Reprezentacje wektorowe	Pojęcia jako wektory (embeddingi)	Uczą się z danych, podobieństwo	Nieczytelne dla człowieka

Wiedza a dane

Dane to pojedyncze fakty („Tweety jest pingwinem”). Wiedza to także **związki** i **reguły** („pingwiny są ptakami”, „ptaki zwykle latają, ale pingwiny nie”). Dzięki nim system odpowie na pytanie, którego nie było w danych: „czy Tweety lata?”.

3. Sieci semantyczne

Sieć semantyczna to graf: **węzły** to pojęcia, **krawędzie** to relacje. Najważniejsza relacja to `jest` (ang. *is-a*) — buduje hierarchię i pozwala **dziedziczyć** cechy. Cechę szukamy najpierw w danym pojęciu, a jeśli jej nie ma — u przodków. Dzięki temu możemy zapisać **wyjątek**: pingwin pływa, choć ptaki latają.



Sieć semantyczna z przykładu 1 — wyjątek (pingwin pływa) nadpisuje cechę dziedziczoną z „ptak”

Przykład 1 — sieć semantyczna z dziedziczeniem

Klasa przechowuje trójki (podmiot, relacja, obiekt). Metoda `przodkowie` przechodzi graf wszerz po relacji `jest`, a `cecha` zwraca wartość własną albo odziedziczoną i informację, skąd pochodzi.

siec_semantyczna.py

```python
"""Sieć semantyczna: wiedza jako graf trójek (podmiot, relacja, obiekt) z dziedziczeniem przez 'jest'."""
from collections import defaultdict


class SiecSemantyczna:
    def __init__(self):
        self.krawedzie = defaultdict(list)          # podmiot -> [(relacja, obiekt)]

    def dodaj(self, podmiot, relacja, obiekt):
        self.krawedzie[podmiot].append((relacja, obiekt))

    def przodkowie(self, pojecie):
        """Wszystkie pojęcia nadrzędne osiągalne relacją 'jest' (przeszukiwanie wszerz)."""
        wynik, kolejka = [], [pojecie]
        while kolejka:
            biezace = kolejka.pop(0)
            for relacja, obiekt in self.krawedzie[biezace]:
                if relacja == "jest" and obiekt not in wynik:
                    wynik.append(obiekt)
                    kolejka.append(obiekt)
        return wynik

    def cecha(self, pojecie, relacja):
        """Wartość cechy: najpierw własna, potem dziedziczona po przodkach."""
        for kandydat in [pojecie] + self.przodkowie(pojecie):
            for r, obiekt in self.krawedzie[kandydat]:
                if r == relacja:
                    return obiekt, kandydat
        return None, None


if __name__ == "__main__":
    s = SiecSemantyczna()
    s.dodaj("ptak", "jest", "zwierzę")
    s.dodaj("ptak", "porusza_się", "lata")
    s.dodaj("ptak", "pokrycie", "pióra")
    s.dodaj("pingwin", "jest", "ptak")
    s.dodaj("pingwin", "porusza_się", "pływa")      # wyjątek nadpisuje cechę ogólną
    s.dodaj("wróbel", "jest", "ptak")
    s.dodaj("zwierzę", "oddycha", "tak")

    for zwierze in ["wróbel", "pingwin"]:
        for rel in ["porusza_się", "pokrycie", "oddycha"]:
            wartosc, skad = s.cecha(zwierze, rel)
            print(f"{zwierze:8} {rel:12} = {wartosc:6} (odziedziczone z: {skad})")
    print("przodkowie pingwina:", s.przodkowie("pingwin"))

```

kod/siec_semantyczna.py

```
$ python3 -B kod/siec_semantyczna.py
wróbel   porusza_się  = lata   (odziedziczone z: ptak)
wróbel   pokrycie     = pióra  (odziedziczone z: ptak)
wróbel   oddycha      = tak    (odziedziczone z: zwierzę)
```

```
pingwin  porusza_się  = pływa  (odziedziczone z: pingwin)
pingwin  pokrycie     = pióra  (odziedziczone z: ptak)
pingwin  oddycha      = tak    (odziedziczone z: zwierzę)
przodkowie pingwina: ['ptak', 'zwierzę']
```

Zadanie 1 (15 min)

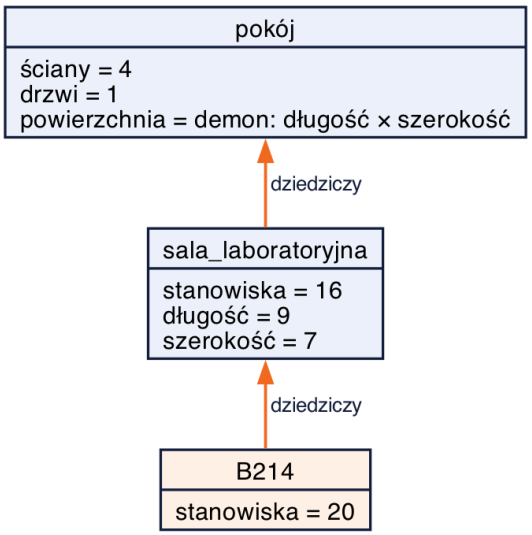
Rozbuduj sieć o gałąź **ssaki**: `ssak jest zwierzę`, `ssak pokrycie sierść`, `nietoperz jest ssak`, `nietoperz porusza_się lata`, `delfin jest ssak`, `delfin porusza_się pływa`. Wypisz wszystkie cechy nietoperza i delfina z informacją, skąd pochodzą.

Zadanie 2 (15 min)

Dodaj metodę `czy_jest(a, b)`, która odpowiada, czy pojęcie `a` jest (bezpośrednio lub pośrednio) rodzajem `b`. Sprawdź: `czy_jest("pingwin", "zwierzę")` → `True`, `czy_jest("zwierzę", "ptak")` → `False`. Dodaj zabezpieczenie przed cyklem w grafie (np. błędnie wpisane `zwierzę jest ptak`).

4. Ramki (frames)

Ramka (Marvin Minsky, 1974) opisuje **stereotypowy obiekt**. Składa się ze **slotów** (cech), które mogą mieć wartości konkretne, **domyślne** (dziedziczone po ramce ogólniejszej) albo **demony** — procedury uruchamiane wtedy, gdy wartość jest potrzebna (*if-needed*) lub gdy się zmienia (*if-added*).



Ramki z przykładu 2 — sloty, wartości domyślne, nadpisanie i demon if-needed

Przykład 2 — ramki z wartościami domyślnymi i demonem

Ramka `B214` dziedziczy po `sala_laboratoryjna`, a ta po `pokój`. Liczba ścian pochodzi z ramki ogólnej, liczba stanowisk jest nadpisana, a powierzchnia liczona przez demona z wymiarów sali.

ramki.py

```python
"""Ramki (frames) Minsky'ego: sloty, wartości domyślne, dziedziczenie i demony 'if-needed'."""


class Ramka:
    def __init__(self, nazwa, rodzic=None, **sloty):
        self.nazwa, self.rodzic, self.sloty = nazwa, rodzic, dict(sloty)

    def znajdz(self, slot, kontekst):
        """Szuka slotu w tej ramce, potem u przodków; demon liczy wartość dla ramki 'kontekst'."""
        if slot in self.sloty:
            wartosc = self.sloty[slot]
            return wartosc(kontekst) if callable(wartosc) else wartosc
        if self.rodzic:
            return self.rodzic.znajdz(slot, kontekst)
        raise KeyError(f"{kontekst.nazwa}: brak slotu '{slot}'")

    def __getitem__(self, slot):
        return self.znajdz(slot, self)


pokoj = Ramka("pokój", sciany=4, drzwi=1,
              powierzchnia=lambda r: r["dlugosc"] * r["szerokosc"])      # demon if-needed
sala = Ramka("sala_laboratoryjna", rodzic=pokoj, stanowiska=16, dlugosc=9, szerokosc=7)
b214 = Ramka("B214", rodzic=sala, stanowiska=20)                         # nadpisanie wartości domyślnej

if __name__ == "__main__":
    for slot in ["sciany", "drzwi", "stanowiska", "powierzchnia"]:
        print(f"B214.{slot:12} = {b214[slot]}")

```

kod/ramki.py

```
$ python3 -B kod/ramki.py
B214.sciany       = 4
B214.drzwi        = 1
B214.stanowiska   = 20
B214.powierzchnia = 63
```

Zadanie 3 (25 min)

Opisz ramkami **pojazdy**: ramka ogólna `pojazd` (koła=4, napęd=spalinowy, zasięg — demon liczący `pojemność_zbiornika / spalanie * 100`), ramki `samochód_osobowy`, `motocykl` (koła=2) i `auto_elektryczne` (napęd=elektryczny, własny demon zasięgu z pojemności baterii). Utwórz dwa konkretne egzemplarze i wypisz ich sloty. Dodaj demon *if-added*, który odrzuca ujemną wartość spalania.

5. Logika i reguły — wnioskowanie w przód

W logice predykatów fakt zapisujemy jako `rodzic(Anna, Bartek)`, a regułę jako implikację: `dziadek(X, Z) ← rodzic(X, Y) ∧ rodzic(Y, Z)`. **Wnioskowanie w przód** (*forward chaining*) stosuje reguły do znanych faktów tak długo, aż nie powstaje nic nowego. **Wnioskowanie wstecz** (*backward chaining*) zaczyna od pytania i szuka faktów, które je potwierdzą — tak działa Prolog.

> **Przykład 3 — fakty, reguły i wnioskowanie w przód**
>
> Program powtarza rundy wnioskowania do osiągnięcia punktu stałego i odpowiada na zapytanie o wnuki Anny.

logika.py

```python
"""Wiedza jako fakty i reguły (logika predykatów, klauzule Horna) + wnioskowanie w przód."""

fakty = {("rodzic", "Anna", "Bartek"), ("rodzic", "Bartek", "Celina"), ("rodzic", "Bartek", "Darek")}


def krok(fakty):
    """Jedna runda wnioskowania: stosuje reguły do znanych faktów i zwraca tylko nowe fakty."""
    nowe = set()
    rodzice = [(x, y) for (p, x, y) in fakty if p == "rodzic"]
    for (x, y) in rodzice:
        for (y2, z) in rodzice:
            if y == y2:                       # dziadek(X, Z) <- rodzic(X, Y) ∧ rodzic(Y, Z)
                nowe.add(("dziadek", x, z))
        for (x2, z) in rodzice:
            if x == x2 and y != z:            # rodzeństwo(Y, Z) <- rodzic(X, Y) ∧ rodzic(X, Z) ∧ Y ≠ Z
                nowe.add(("rodzenstwo", y, z))
    return nowe - fakty


if __name__ == "__main__":
    runda = 1
    while True:
        nowe = krok(fakty)
        if not nowe:
            break
        print(f"runda {runda}: {sorted(nowe)}")
        fakty |= nowe
        runda += 1
    print("zapytanie dziadek(Anna, ?) ->", sorted(z for p, x, z in fakty if p == "dziadek" and x == "Anna"))

```

kod/logika.py

```
$ python3 -B kod/logika.py
runda 1: [('dziadek', 'Anna', 'Celina'), ('dziadek', 'Anna', 'Darek'), ('rodzenstwo',
'Celina', 'Darek'), ('rodzenstwo', 'Darek', 'Celina')]
zapytanie dziadek(Anna, ?) -> ['Celina', 'Darek']
```

Zadanie 4 (20 min)

Dodaj regułę `pradziadek(X, W) ← dziadek(X, Z) ∧ rodzic(Z, W)` oraz fakt `rodzic(Celina, Ewa)`. Ile rund potrzeba teraz do osiągnięcia punktu stałego? Wyjaśnij dlaczego.

Zadanie 5 (20 min)

Zaimplementuj **wnioskowanie wstecz** dla zapytania `dziadek(Anna, ?)`: funkcja rekurencyjna, która dla celu szuka faktu wprost albo reguły, której wniosek pasuje do celu, i sprawdza jej przesłanki. Porównaj liczbę sprawdzonych faktów z wnioskowaniem w przód.

6. Projekt semestralny

Zasady zaliczenia — projekt, bez kolokwiów

Przedmiot zaliczamy **projektem** oddanym w Moodle i zaprezentowanym na ostatnich zajęciach (12.01.2027). Zespoły 2–3 osobowe. Projekt to działający program w Pythonie, który wykorzystuje **co najmniej dwie** metody z semestru (np. reprezentacja wiedzy + reguły, algorytm genetyczny, logika rozmyta, sieć neuronowa). Oddajesz: repozytorium Git z instrukcją uruchomienia (`README.md`), krótki raport (3–5 stron: problem, metoda, wyniki, wnioski) i prezentację 10 minut.

Przykładowe tematy:

- system ekspercki doradzający wybór laptopa (reguły + wnioskowanie wstecz),
- planer rozkładu zajęć optymalizowany algorytmem genetycznym,
- sterownik rozmyty klimatyzacji lub parkowania,
- klasyfikator gatunków muzyki siecią neuronową z bazą wiedzy o gatunkach,
- chatbot FAQ uczelni z ontologią pojęć (sieć semantyczna) i dopasowaniem pytań.

Na koniec zajęć (40 min)

1. Utwórzcie zespół i wybierzcie temat (może być własny — do akceptacji prowadzącego).
2. Załóżcie repozytorium, dodajcie prowadzącego, wrzućcie `README.md` z tematem i składem zespołu.
3. Wklejcie w Moodle link do repozytorium i jednozdaniowy opis problemu.

7. Do samodzielnego przeczytania

- M. Flasiński, *Wstęp do sztucznej inteligencji*, PWN 2011 — rozdz. o reprezentacji wiedzy.
- S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach* — rozdz. „Knowledge Representation”.
- B. Stefanowicz, *Sztuczna inteligencja: systemy eksperckie* — rozdz. o regułach i wnioskowaniu.